

プログラミング④

(アルゴリズム：ソートと近似)

2026 年度高 2 情報

2026/06/06

アルゴリズムの2つのアプローチ

目的を達成するための「手順」の多様性

これまで学んだ基本構文（for 文, if 文, 配列）を組み合わせ、複雑な問題を解く手順（アルゴリズム）を作る

- 決まった手順通りに動かし、必ず1つの正しい「厳密解」を導く。
→ バブルソート と クイックソート
- あえて「乱数」を利用し、実験を繰り返すことで正解に近づける。
→ モンテカルロ法による円周率の近似

バブルソート（基本のソート）

隣り合う要素を比べるソート

- 配列の先頭から、**隣り合う2つの要素**を比較する.
- 大小関係が逆であれば、その場で要素を「入れ替える」.
- これを末尾まで繰り返すと、一番大きな値が一番後ろに移動する
- データ数を N とすると、計算量は $O(N^2)$ となる.

バブルソートの実装例

```
public class BubbleSortEx {  
    public static void BubbleSort(int[] arr) {  
        int n = arr.Length;  
        for (int i = 0; i < n - 1; i++) {  
            for (int j = 0; j < n - 1 - i; j++) {  
                if (arr[j] > arr[j + 1]) {  
                    int temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp;  
                }  
            }  
            System.Console.WriteLine((i + 1) + " 回目: " + string.Join(",", arr));  
        }  
    }  
    public static void Main() {  
        int[] data = { 5, 3, 8, 4, 2 };  
        System.Console.WriteLine("初期: " + string.Join(",", data));  
        BubbleSort(data);  
    }  
}
```

クイックソート（高速なソート）

効率的なソート

バブルソートのように全体を愚直に比べるのではなく、問題を小さく分けて解く手法

1. 配列の中から基準となる値（**ピボット**）を1つ選ぶ.
2. 左端からピボット以上の値, 右端からピボット以下の値を探し, 見つけたら**入れ替える**.
3. この操作によって, ピボットより「小さいグループ」と「大きいグループ」に分割する.
4. 分割された各グループに対して, 再び同じ処理を再帰的に繰り返す.

平均的な計算量は $O(N \log_2 N)$. 圧倒的に高速.

計算量（オーダー）の圧倒的な差

データの数が増大になったときの処理時間

アルゴリズムの工夫によって、コンピュータの処理時間は劇的に変わる

データ数 N	バブルソート $\frac{N(N-1)}{2}$	クイックソート $N \log_2 N$
10^3 件	4.995×10^5 回	約 9.966×10^3 回
10^5 件	4.99995×10^9 回	約 1.661×10^6 回

クイックソートの実装例

```
public class QuickSortEx {  
    public static void QuickSort(int[] arr, int left, int right) {  
        if (left >= right) return;  
        int pivot = arr[(left + right) / 2]; // 中央の値を基準に  
        int i = left, j = right;  
        while (i <= j) {  
            while (arr[i] < pivot) i++;  
            while (arr[j] > pivot) j--;  
            if (i <= j) {  
                int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;  
                i++; j--;  
            }  
        }  
        System.Console.WriteLine($"基準{pivot}: " + string.Join(",", arr));  
        QuickSort(arr, left, j); // 小さいグループを再帰  
        QuickSort(arr, i, right); // 大きいグループを再帰  
    }  
    public static void Main() {  
        int[] data = { 5, 3, 8, 4, 2, 7, 1 };  
        System.Console.WriteLine("初期: " + string.Join(",", data));  
        QuickSort(data, 0, data.Length - 1);  
    }  
}
```

モンテカルロ法とは？

乱数が解を導くアプローチ

- 厳密な数式や手順で直接解くのが難しい問題に対して、**大量の「乱数」を用いたシミュレーション**を行い、確率的に近似解（だいたいの正しい値）を求める手法
- 実験の回数（サンプリング数）を増やせば増やすほど、本来の正しい解に確率的に近づいていく（大数の法則）

モンテカルロ法による円周率 π の近似

発想

一辺の長さが 1 の正方形（面積 1）の中に、ランダムに点 (x, y) を打つ ($0 \leq x, y \leq 1$)

- 原点からの距離が 1 以内, つまり $x^2 + y^2 \leq 1$ を満たす点は, 半径 1 の**四半円の内部**に入る.
- 四半円の面積は $\frac{\pi}{4}$ なので, 点が円内に入る確率は $\frac{\pi}{4}$ になる.

$$\frac{\text{円内に入った点の数}}{\text{すべての点の数}} \approx \frac{\pi}{4} \implies \pi \approx 4 \times \frac{\text{円内に入った点の数}}{\text{すべての点の数}}$$

円周率近似の実装例

```
public class MonteCarlo {  
    public static double EstimatePi(int totalPoints) {  
        System.Random rand = new System.Random();  
        int insideCount = 0;  
        for (int i = 0; i < totalPoints; i++) {  
            double x = rand.NextDouble(); // 0.0以上1.0未満の乱数  
            double y = rand.NextDouble();  
            if (x * x + y * y <= 1.0) {  
                insideCount++;  
            }  
        }  
        // 先に4.0を掛けることでdouble型の計算にする  
        return 4.0 * insideCount / totalPoints;  
    }  
  
    public static void Main() {  
        System.Console.WriteLine(EstimatePi(100));  
    }  
}
```

paiza.io/ja では, 10^7 回くらいが限界なようです.

問題1

問題1

クイックソートは平均して非常に高速だが、基準値（ピボット）の選び方によっては、最悪の場合バブルソートと同じ $O(N^2)$ まで遅くなってしまうという問題点がある。

この「最悪のケース」をできるだけ避けるために、どのようなピボットの選び方の工夫が考えられるか、アイデアを提案したり、調べてみたりせよ。

最後の2枚のスライドが実質的な解答である。見ないで熟考せよ。

問題2

問題2

前ページの関数 `EstimatePi` において, 実験回数 `totalPoints` を「100」「10,000」「1,000,000」と増やしていったとき

- (1) 得られる円周率の近似値の「精度」
- (2) コンピュータの「計算時間 (処理の重さ)」

はそれぞれどのように変化するか. プログラムで精度や計算時間を計算するコードをかけ.

次のスライドとその次のスライドに使う道具 (タイマーなど) のヒントがある.

問題2へのアプローチ

① 処理時間の計測（ストップウォッチ機能）

`System.Diagnostics.Stopwatch` を使って時間を計測します。まずはストップウォッチ本体（変数 `sw`）を準備します。

- `Stopwatch sw = new Stopwatch();` で準備
- `sw.Start();` で計測スタート
- `sw.Stop();` で計測ストップ
- `sw.ElapsedMilliseconds` でかかった時間（ミリ秒）を取り出す

問題2へのアプローチ

② 正確な円周率との誤差（精度）

C#にはあらかじめ円周率 `System.Math.PI` が用意されているこれと自分の計算結果との「差の絶対値」を `System.Math.Abs()` を使って計算すれば、どれくらいズレているか（誤差）が分かる

その他の代表的なソートアルゴリズム

ヒープソート

- データを「ヒープ（木構造）」という構造に当てはめて並び替える
- 最悪の場合でも計算量が $O(N \log_2 N)$ で収まる
- マージソートと異なり、追加のメモリ領域をほとんど消費しない
- イントロソートの中で、計算量を保証するための切り替え先として使われる

マージソート

- 要素が1つになるまで分割し、順序を保ちながら結合していく
- 最悪の場合でも計算量が $O(N \log_2 N)$ で収まる
- 結合の処理を行うため、追加のメモリ領域を消費する

問題1の解答例：ハイブリッド・ソート

イントロソート (IntroSort)

- C#やC++の標準ソートとして採用されている
- 最初はクイックソートで並び替えを行う
- 分割が偏り再帰が深くなりすぎた場合、ヒープソートに切り替える
- 要素数が少なくなったら挿入ソートで処理を完了させる

ティムソート (Timsort)

- Python や Java の標準ソートとして採用されている
- マージソートと挿入ソートを組み合わせたアルゴリズム
- データ内にすでにソートされている部分を見つけて利用する
- 現実のデータに対して処理を省略しやすく、高速に動作する